NixOS and the Congruence Problem

Sameer Gupta

2026

Reproducible systems, and Congruence

This is verifiable with `nix build --rebuild`, which builds twice and compares. Deployment via `deploy-rs` or `nixos-rebuild` over SSH performs an atomic generation switch, with automatic rollback on failure.

> Contrast this with Bazel, which sandboxes build actions but typically inherits its toolchain from the host system. Two engineers on different Linux distributions running the same Bazel build may produce different binaries. Bazel enforces congruence within the build graph; Nix enforces it all the way down to the metal. Combining both — via `rules_nixpkgs` — is increasingly common precisely because they address different layers.

Code as Infrastructure, or Something Else?

Dellaiera's 2024 thesis [1] frames the problem at the artifact level: can you take source code and deterministically produce identical binaries, regardless of when or where you build? This is a real and important question, particularly for supply-chain security. But it operates one level below what NixOS offers.

NixOS does not just make builds reproducible. It makes the entire system — kernel, services, application, deployment pipeline — a single evaluated expression in the same language, stored in the same content-addressed store, governed by the same hermetic rules.

> "Infrastructure as Code" is the wrong frame. It implies that infrastructure is the primary thing, and code is the tool you use to manage it. "Code as Infrastructure" is closer, but it still implies a separation between the two. NixOS makes that separation a **category error**.

The compiler is a derivation. The OS is a derivation. Your application is a derivation. The deployment pipeline is a derivation. There is no privileged "real" infrastructure that the code merely describes. There is only the evaluation of expressions and the content-addressed store that holds their results.

[1] Dellaiera, P. (2024). *Reproducibility in Software Engineering*. Master's thesis, University of Mons. Supervised by Tom Mens. Zenodo v6, February 2026. DOI 10.5281/zenodo.18701980

Conclusion

Traugott and Brown wrote in 2002 that achieving congruence was an unsolved problem in infrastructure management. Two decades later, NixOS is the most complete practical answer to that problem that exists.

> You do not need to argue your team into a philosophical position. You write a `flake.nix`, commit a `flake.lock`, and the congruence is structural. **Drift has nowhere to live.**

Its design did not emerge from the IaC tradition — it emerged from functional programming and formal package management — and that lineage shows. It does not try to converge a running system toward a specification. It makes the specification and the system the same object.